Lecture 12

21 October 2026 19:54

Now for more good training we will introduce batch-norm (i.e. batch normalization)

1. The Exact BatchNorm1d from Makemore

Here is the exact scratchpad implementation Karpathy pastes into the cell:

Python

```
class BatchNorm1d:

  def __init__(self, dim, eps=1e-5):
    self.eps = eps
    self.gamma = torch.ones(dim)   # learnable scale
    self.beta = torch.zeros(dim)      # learnable shift

  def __call__(self, x):

    # Calculate mean and variance across the BATCH (dimension 0)

    xmean = x.mean(0, keepdim=True)  # batch mean
    xvar  = x.var(0, keepdim=True          # batch variance

    # Normalize to zero mean and unit variance

    xhat = (x - xmean) / torch.sqrt(xvar + self.eps)
    self.out = self.gamma * xhat + self.beta
    return self.out

  def parameters(self):
    return [self.gamma, self.beta]
```

Explaination of the code:

1. The Big Goal: Why normalize at all?

As numbers pass through layers of a neural network:

- Some numbers grow uncontrollably huge (like $+500$)
- Some numbers shrink to near zero (like 0.0001)

When numbers get out of control, **gradients explode or vanish**, and training crashes.

V. IMP

Normalization is like a reset button: it pulls all numbers back into a nice, healthy bell curve where:

- The **average (mean)** is 0
- The **spread (variance / standard deviation)** is 1 (Most numbers will comfortably sit between -2 and $+2$).

1. What is dim?

dim is the number of features (e.g., 100 or 32). Every feature gets its own scale and shift parameters.

2. What is eps ($10^{-5} = 0.00001$)?

It's a tiny safety number. In the math, we divide by $\sqrt{variance}$. If variance happens to be 0, dividing by zero would crash the computer! Adding eps ($0 + 0.00001$) prevents dividing by zero.

3. What are gamma and beta?

- What if the neural network *doesn't* want mean=0 and variance=1? What if it *needs* a feature to be large or positive?
- gamma (starts at 1) allows the network to **scale** the numbers.
- beta (starts at 0) allows the network to **shift** the numbers.
- Because they are trainable, the network has the power to undo or adjust the normalization if it wants to!

The Forward Pass (__call__):

Python

```
  def __call__(self, x):

    # Step A: Calculate mean and variance

    xmean = x.mean(0, keepdim=True)    # batch mean
    xvar  = x.var(0, keepdim=True)          # batch variance
```

V imp :
4. What does x.mean(0, keepdim=True) do?

Suppose our batch x has shape (32, 100): 32 examples, each having 100 features.

1. V V IMP:

dim=0 means: **average across the 32 rows (down each column)**.

2. V IMP : Result: shape (1, 100)—one average value for each of the 100 features.
3. keepdim=True keeps the 2D shape (1, 100) so PyTorch can easily subtract it from (32, 100) using broadcasting.

```
# Step B: The Standard Z-Score Formula
xhat = (x - xmean) / torch.sqrt(xvar + self.eps)
```

V IMP:
5. How does this normalize?

This is the classic statistics formula: $z = \frac{x - \mu}{\sigma}$

1. **x - xmean)**: Centers the data. If the average score was 80, a score of 80 becomes $80 - 80 = 0$.

   V IMP
   2: / torch.sqrt(xvar + self.eps): Divides by the spread (standard deviation σ). If scores are wildly spread out, dividing compresses them down so the variance becomes exactly 1.0.

2. Now, xhat is perfectly normalized!

```
# Step C: Scale and Shift
self.out = self.gamma * xhat + self.beta
return self.out
```

6. Applying gamma and beta:

- Initially: [Equation] .
- As the model trains, it can tweak gamma and beta to shift the distribution wherever it helps prediction most.

In Plain English:

1. Find the average and spread across all examples in the batch.
2. Center everything to **0** and scale the spread to **1**.
3. Let the network slightly scale (gamma) and shift (beta) the result if needed.

Why do we use scaling and shifting :

Think of **Scaling** and **Shifting** like the controls on a stereo or guitar amplifier:

- **Scaling (γ) = Volume Knob** (makes the signal louder/wider or quieter/narrower).
- **Shifting (** [Equation] **) = Pitch / Position Slider** (moves the entire signal higher or lower).

Let's see exactly how this works with real numbers.

1. What is Scaling? ([Equation] — Gamma)

Scaling means multiplying by a number.

It controls **how spread out** the numbers are (the spread/variance).

Suppose our normalized numbers are centered around 0: [Equation]

- If [Equation] :
  [-1.0, 0.0, 1.0] * 1.0 ⟶ [-1.0, 0.0, 1.0] *(unchanged)*
- If [Equation] **(Scale Up)**:
  [-1.0, 0.0, 1.0] * 3.0 ⟶ [-3.0, 0.0, 3.0] *(spread is 3× wider)*
- If [Equation] **(Scale Down)**:
  [-1.0, 0.0, 1.0] * 0.5 ⟶ [-0.5, 0.0, 0.5] *(spread is narrower)*

2. What is Shifting? ([Equation] — Beta)

Shifting means adding a number.

It moves the **center (the average)** left or right along the number line.

Take that same [-1.0, 0.0, +1.0]:

- If [Equation] :
  [-1.0, 0.0, 1.0] + 0.0 ⟶ [-1.0, 0.0, 1.0] *(center stays at 0)*

- If [Equation] **(Shift Right)**:
  [-1.0, 0.0, 1.0] + 10.0 ⟶ [9.0, 10.0, 11.0] *(center is now 10!)*

- If [Equation] **(Shift Left)**:
  [-1.0, 0.0, 1.0] - 5.0 ⟶ [-6.0, -5.0, -4.0] *(center is now -5!)*

3. Putting them together: How it's done

The formula is literally the simple equation of a straight line you learned in school ([Equation]):

[Equation]

Let's trace a full example with [Equation] and [Equation] :

[Equation] [Equation] [Equation]

4. What are [Equation] and [Equation] in PyTorch?

They are **trainable parameters** (weights), just like the weights in nn.Linear:

```
python
self.gamma = torch.ones(dim)   # Starts at 1.0
self.beta  = torch.zeros(dim)  # Starts at 0.0
```

1. **At the start of training:**
   [Equation]
   It does nothing initially! It just passes the clean, normalized data through.

2. **During backpropagation:**
   The optimizer calculates gradients for [Equation] and [Equation] . If changing the scale or shift helps lower the loss, the optimizer updates them!

5. Why do we need this? (Why not just keep mean=0 and variance=1 forever?)

Because **forcing** numbers to stay strictly at mean=0 might hurt the network!

- **Example with ReLU:**
  Remember that ReLU(x) turns all negative numbers to **0**. If our numbers are strictly centered at 0 with half of them negative, **ReLU will kill 50% of the information every single time!**

- By having [Equation] (shift), the network can say:
  "Hey, for this specific layer, I'm going to shift [Equation] *to* [Equation] *so that most numbers are positive and survive ReLU!"*

---------- we do batch norm ----------

You got the main purpose: it acts as a **stabilizer** so activations and weights never explode or vanish during training.
Just keep this tiny pairing clear in your mind:

Statistic	What it represents	What controls it	Action
Mean ([Equation])	The **center** of the data	[Equation] (Beta)	**Shifting** (moves left/right)
Variance ([Equation])	The **spread / width** of the data	[Equation] (Gamma)	**Scaling** (stretches or squeezes)

The 2-Step Dance:

1. **Step 1 (Normalize):**
   - Subtract the **mean** ⟶ centers data at 0.
   - Divide by $\sqrt{variance}$ ⟶ scales spread to 1.

2. **Step 2 (Restore Freedom):**
   - Multiply by [Equation] ⟶ network chooses its own **scale**.
   - Add [Equation] ⟶ network chooses its own **shift**.

The Bridge to our Transformer:

In our Transformer, we do this **exact same math**, but instead of doing it across the batch (BatchNorm), we do it across the 32 features of each token (**LayerNorm**).
And in PyTorch, you don't even have to write the math by hand—PyTorch handles the mean, variance, [Equation] , and [Equation] automatically with just:

```
python
self.ln = nn.LayerNorm(n_embd)
```

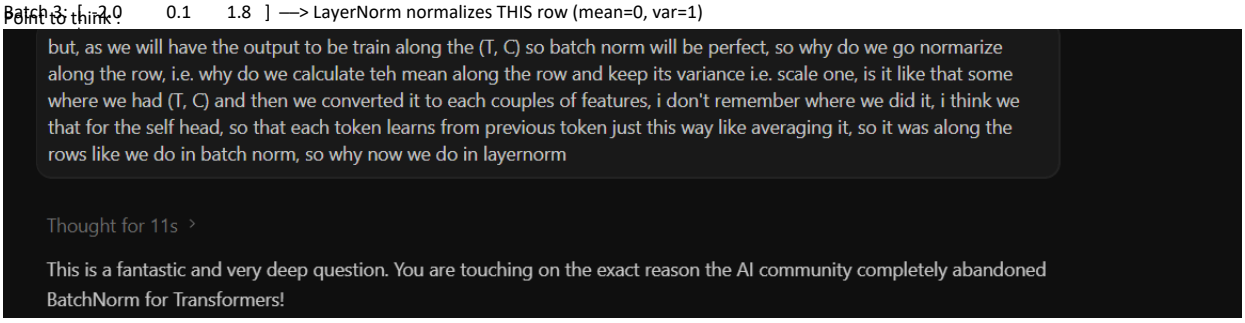
V·V·IMP

To understand why we are doing layer norm

Summary :
So, the batch norm is saving of the weights which will be used during deep network training of our model, so to keep it safe and not lossing it, we do scaling i.e. (keeping mean at = 0) and shifting (i.e. adding a number) and this is the formula of how the weights will be safe, and process to apply it is this : gamma (scaling) * normarize + beta (shifting) used in the network.

```
        Feature 1   Feature 2   Feature 3
Batch 1:  [  1.2       -0.4       3.1  ]  —> LayerNorm normalizes THIS row (mean=0, var=1)
Batch 2:  [  0.5        2.1      -1.0  ]  —> LayerNorm normalizes THIS row (mean=0, var=1)
Batch 3:  [ -2.0        0.1       1.8  ]  —> LayerNorm normalizes THIS row (mean=0, var=1)
```

Point to think :



This is a fantastic and very deep question. You are touching on the exact reason the AI community completely abandoned BatchNorm for Transformers!

Let's untangle the confusion between **Attention (Time T)** and **Normalization (Features C)**.

1. First, clear up the confusion: Attention vs. Normalization

You mentioned:
> *"I think we did that for the self head, so that each token learns from previous token just this way like averaging it..."*

Yes! You are remembering wei @ v in Attention!

- In Attention, a token at time t looks across **Time (T)** to average information from previous tokens.
- **Attention is communication across tokens.**
- **LayerNorm is NOT communication.** LayerNorm does not share information between tokens at all!

2. The Tensor Shape: [Equation]

Every tensor in our model has 3 dimensions:
- [Equation] **(Batch):** Independent sentences (e.g., 32 different text snippets).
- [Equation] **(Time):** Positions in the sentence (e.g., 8 characters: ['t', 'h', 'e', ' ', 'c', 'a', 'r', ' ']).
- [Equation] **(Channels):** The **32 features** describing that one character.

```
Token at position 3:
[ f_0, f_1, f_2, ... f_31 ]  <-- 32 features ©
```

3. Why BatchNorm is a disaster for Transformers

In **BatchNorm**, you normalize down the **Batch dimension (B)**: It asks: *"What is the average of Feature #0 across all 32 different sentences in the batch?"*

This creates **two fatal problems** for text: ← Part:1

Problem A: Sentences leak into each other!

V IMP :
1. Batch 1 might be a scene from *Romeo and Juliet*.
2. Batch 2 might be a scene from *Hamlet*
3. In BatchNorm, the math of Romeo's sentence **directly depends on Hamlet's sentence** to calculate the batch mean! That makes zero sense—different sentences in a batch are supposed to be completely independent.

Problem B: Text Generation crashes (B = 1)! ← Part: 2

When you generate text using:

```
context = torch.zeros((1, 1), dtype=torch.long) # Batch size B = 1!
model.generate(context, max_new_tokens=500)
```

V IMP :
1. During generation, your batch size is $B = 1$ (you are generating one story).
2. How can you compute a "batch average" across sentences when there is only **1 sentence**?
3. In BatchNorm, the variance of 1 number is 0, leading to division by zero and garbage output.

4. Why LayerNorm is PERFECT for Transformers 🏆

In **LayerNorm**, we normalize across the **Channel dimension (C)**:

For **every single token**, we take its **32 features**: [Equation]

1. Calculate the mean of **those 32 numbers**.
2. Calculate the variance of **those 32 numbers**.
3. Normalize **those 32 numbers** so they have mean=0 and variance=1.

Sentence 1, Token 1: [32 features] ——> Normalized ALONE!
Sentence 1, Token 2: [32 features] ——> Normalized ALONE!
Sentence 2, Token 1: [32 features] ——> Normalized ALONE!

The Big Difference in One Sentence:

- **BatchNorm:** *"Hey all 32 sentences in the batch, let's pool our numbers together and find the average!"* ❌ (Messy, leaks data, breaks when $B = 1$).
- **LayerNorm:** *"Every single token cleans its own room (its own 32 features) in total privacy."* (Clean, independent, works whether $B = 1$ or [Equation]).

Now, again the this line are introduced:
X = x + self.sa(self.ln1(x))
X = x + self.ffwd(self.ln2(x))

And in the bigram model in __init__:

In the block, add this line nn.LayerNorm(n_embd)

The loss In the val is now 2.08 something which has been improved